

BANCO DE INCIDENTES DE SEGURANCA

Guia pratico para entender o projeto, o banco relacional e as consultas

5	6	2	5
tabelas	eventos	incidentes	consultas

O objetivo deste material e explicar o raciocinio por tras de cada arquivo. Assim, voce nao apenas possui um projeto pronto: consegue contar para um recrutador o problema, as decisoes e os resultados.

O que foi construido

Um banco SQLite que registra usuarios, enderecos IP, eventos de autenticao e incidentes. Os dados simulam tentativas de login e permitem encontrar um possivel ataque de forza bruta: tres falhas do IP 192.168.1.25 em apenas 15 segundos.

Arquivos do projeto

Arquivo	Responsabilidade
schema.sql	Cria as tabelas, chaves e regras de integridade.
dados.sql	Insere dados ficticios para demonstracao.
consultas.sql	Transforma registros em informacoes para investigacao.
incidentes.db	Banco local gerado. Nao precisa ser enviado ao GitHub.
README.md	Apresenta o projeto ao recrutador e ensina como executar.

1. Modelagem: como as tabelas se conectam

Tabela	O que representa	Relacionamento
usuarios	Pessoas que realizam autenticacoes.	1 usuario pode gerar varios eventos.
enderecos_ip	Origens de rede observadas.	1 IP pode aparecer em varios eventos.
eventos	Cada tentativa de autenticacao.	Liga usuario e IP por chaves estrangeiras.
incidentes	Casos abertos para investigacao.	Pode reunir varios eventos.
incidentes_eventos	Tabela de ligacao.	Resolve a relacao muitos-para-muitos.

Por que existem chaves?

A chave primaria identifica cada linha. A chave estrangeira garante que um evento nao aponte para um usuario ou IP inexistente. UNIQUE evita repeticoes indevidas. CHECK limita valores aceitos, como SUCESSO ou FALHA.

Exemplo mental

O evento de id 2 nao precisa repetir o nome Maria Oliveira nem o texto do IP. Ele guarda usuario_id = 2 e endereco_ip_id = 2. As consultas usam JOIN para recuperar os dados completos.

2. Dados de demonstracao

Os INSERTs foram separados do esquema para que a estrutura possa ser reutilizada com outros dados. Todos os nomes, e-mails e enderecos sao ficticios e seguros para um portfolio publico.

```
INSERT INTO eventos (usuario_id, endereco_ip_id, data_hora, tipo, resultado)
VALUES (2, 2, '2026-08-13 08:12:03', 'LOGIN', 'FALHA');
```

Leitura: Maria (usuario 2), usando o IP 192.168.1.25 (IP 2), realizou uma tentativa de login que falhou naquele horario.

3. As cinco consultas

Consulta 1 - Lista detalhada

Usa INNER JOIN para combinar eventos, usuarios e IPs. ORDER BY organiza cronologicamente. E a visao operacional de tudo que aconteceu.

Consulta 2 - Falhas por IP

Filtra apenas FALHA, agrupa por IP e usa COUNT, MIN e MAX. HAVING remove grupos com poucas falhas. Foi assim que encontramos 3 falhas em 15 segundos.

Consulta 3 - Eventos de cada incidente

Percorre a tabela de ligacao incidentes_eventos. Demonstra uma relacao muitos-para-muitos e mostra quais evidencias sustentam cada investigacao.

Consulta 4 - Resumo executivo

Usa LEFT JOIN para manter incidentes mesmo sem eventos, GROUP_CONCAT para reunir IPs e CASE para ordenar gravidades segundo a regra do negocio.

Consulta 5 - Classificacao por usuario

SUM com CASE faz contagem condicional. Cada sucesso ou falha vira 1; os demais casos viram 0. Outro CASE converte o total de falhas em NORMAL, OBSERVAR ou ATENCAO.

4. Entendendo a consulta final

```
SUM(CASE WHEN eventos.resultado = 'FALHA' THEN 1 ELSE 0 END)
```

Para cada evento, o CASE pergunta se o resultado foi FALHA. Se sim, devolve 1; se não, devolve 0. SUM soma esses numeros dentro do grupo de cada usuario.

Por que LEFT JOIN?

Ele preserva usuarios que ainda não possuem eventos. Com INNER JOIN, esses usuarios desapareceriam do relatorio. COALESCE não foi necessario nas contagens porque o CASE já produz zero.

Usuario	Tentativas	Sucessos	Falhas	Classificacao
Maria Oliveira	3	0	3	ATENCAO
Joao Santos	2	1	1	OBSERVAR
Nicolas Marques	1	1	0	NORMAL

5. Como executar do zero

No terminal, dentro da pasta do projeto:

```
sqlite3 incidentes.db ".read schema.sql"
sqlite3 incidentes.db ".read dados.sql"
sqlite3 -header -column incidentes.db ".read consultas.sql"
```

O primeiro comando cria as tabelas. O segundo preenche o banco. O terceiro executa os relatorios com cabecalhos e colunas alinhadas.

6. Como apresentar em uma entrevista

Eu modeliei um banco SQLite para registrar eventos de autenticação e investigar incidentes. Separei estrutura, dados e consultas. Usei chaves estrangeiras para integridade, uma tabela associativa para muitos-para-muitos e consultas com JOIN, GROUP BY, HAVING, agregações e CASE. Nos dados fictícios, o projeto detecta três falhas do mesmo IP em 15 segundos e organiza as evidências por incidente.

7. Proximos exercicios para aprender de verdade

- Alterar o limite da Consulta 2 de 2 para 3 falhas e comparar o resultado.
- Cadastrar um novo usuario e um evento de falha.
- Criar uma consulta que conte incidentes por status.

- Adicionar uma coluna encerrado_em e atualizar um incidente para RESOLVIDO.
- Explicar, sem olhar o código, por que incidentes_eventos possui duas chaves estrangeiras.

Conclusão

O projeto está completo para uma primeira versão de portfólio: modelo coerente, dados seguros, consultas relevantes, validação automatizada e documentação visual. O passo seguinte é publicá-lo no GitHub e usá-lo como evidência concreta do seu aprendizado em SQL e Segurança da Informação.

Glossário rápido

Termo	Significado neste projeto
SELECT	Escolhe as colunas que aparecerão no resultado.
JOIN	Combina linhas de tabelas relacionadas.
WHERE	Filtra linhas antes do agrupamento.
GROUP BY	Forma grupos, como um grupo para cada IP ou usuário.
HAVING	Filtra grupos depois de COUNT, SUM, MIN ou MAX.
CASE	Aplica uma regra condicional dentro da consulta.
FOREIGN KEY	Protege o relacionamento entre duas tabelas.
Muitos-para-muitos	Dois lados podem ter vários registros relacionados; exige tabela de ligação.

Checklist de domínio

Você está pronto para apresentar o projeto quando conseguir: (1) explicar as cinco tabelas; (2) diferenciar WHERE de HAVING; (3) mostrar por que usamos uma tabela associativa; e (4) interpretar o alerta de três falhas em 15 segundos.